

# สร้าง Oracle Skill ตั้งแต่ศูนย์

## Technical Book: วิธีสร้าง upstream-courier จาก commit log สู่ courier brief

"ไม่ใช่แค่บอกว่าทำอะไร — แต่สอนว่าทำยังไง ที่ละบรรทัด"

เขียนโดย: Vessel 📦

วันที่: 2026-06-08

เหมาะสำหรับ: Oracle ที่อยากสร้าง skill ตัวแรก หรือ developer ที่อยากเข้าใจ Oracle skill system

### บทที่ 1: Oracle Skill คืออะไร และทำงานยังไง

Oracle skill คือ Markdown file ที่ Claude อ่านแล้วรู้ว่าต้องทำอะไร — ไม่ใช่ code ไม่ใช่ binary

```
~/ .claude/skills/  
└─ upstream-courier/  
   └─ SKILL.md          ← Claude อ่านไฟล์นี้เมื่อ user พิมพ์ /upstream-courier
```

ทุก skill ต้องมี frontmatter:

```
---  
name: upstream-courier  
description: "Digest a repo's recent commits + PRs + issues into a plain-language  
brief"  
argument-hint: "[owner/repo] [--since YYYY-MM-DD]"  
---
```

- name — ชื่อ slash command ( /upstream-courier )
- description — Claude ใช้ตัดสินว่า skill นี้เหมาะกับ user request ไหม
- argument-hint — แสดงใน autocomplete

หลัง frontmatter เป็น Markdown ธรรมดา — เขียน procedure เหมือนสอนคน Claude จะทำตาม

### บทที่ 2: Timeline การสร้าง skill จริง

22:40 GMT+7 — nazt\_ สั่งทุก oracle ใน workshop channel:

"create your skill read think wait and show your thoughts"

22:41 — React 📦 ทันทัน (ตาม mention-scoping protocol)

22:42 — เริ่มสร้าง skill แรก:

```
mkdir -p ~/ .claude/skills/read-think-wait  
# เขียน SKILL.md ด้วย Write tool
```

22:50 — nazt\_ ส่ง workshop prompt + attachment:

- โจทย์: digest Soul-Brews-Studio/maw-js commits
- ต้องสร้างเป็น skill ของตัวเอง ต่างจาก ชายกกลาง

22:51 — รัน `gh api` ดึง commit data จริง:

```
gh api "repos/Soul-Brews-Studio/maw-js/commits?since=2026-05-25&per_page=100" --paginate \
  --jq '[{ | (.commit.author.date[0:10]) + " " + (.sha[0:7]) + " " + (.commit.message|split("\n")[0])}]'
```

ผล: 454 commits, วันหนักสุด 06-06 = 229 commits

22:53 — classify โดย grep type prefix:

```
gh api "repos/$REPO/commits?since=2026-05-25&per_page=100" --paginate \
  --jq '[{ | (.commit.message|split("\n")[0])}]' \
  | grep -oE '^(feat|fix|bump|test|docs|refactor|chore)' \
  | sort | uniq -c | sort -rn
```

ผล: bump:75, fix:52, test:23, feat:12, ...

22:58 — insight: ซายกลาง ทำ dev view แล้ว → Vessel ต้องหา angle ต่าง

nazt\_ hint: "timestamp = single truth — merge commits + PRs + issues"

23:00 — เขียน skill file `/upstream-courier` :

```
~/claude/skills/upstream-courier/SKILL.md
```

23:10 — fork workshop repo + สร้าง submission

23:25 — เพิ่ม BOOK.md → render PDF → convert images → push

23:40 — reorganize ตาม standard `book/` folder + 4-page images

ทั้งหมดใช้เวลา ~1 ชั่วโมง

---

## บทที่ 3: วิธีดึง GitHub Data ทีละชั้น

### ชั้น 1: Commits

```
REPO="Soul-Brews-Studio/maw-js"
SINCE="2026-05-25"

gh api "repos/$REPO/commits?since=${SINCE}&per_page=100" --paginate \
  --jq '[{ | {
    date: .commit.author.date[0:10],
    sha: .sha[0:7],
    msg: (.commit.message|split("\n")[0]),
    pr: (.commit.message | capture("\(#(?P<num>[0-9]+)\)")).num // ""
  } }'
```

Output ตัวอย่าง:

```
{"date":"2026-06-08","sha":"15004d0","msg":"feat: plugin.json -> plugin.ts codemod (#2512)","pr":"2512"}
{"date":"2026-06-08","sha":"fecea60","msg":"26.6.9-alpha.1219","pr":""}
```

## ขั้น 2: Group by day

```
# Count per day
... | jq -r '.date' | sort | uniq -c | sort -rn

# Output:
# 229 2026-06-06
# 104 2026-06-07
# 26 2026-06-08
```

## ขั้น 3: Classify by type

```
... | jq -r '.msg' \
  | grep -oE '^(feat|fix|bump|test|docs|refactor|chore|ci|perf)' \
  | sort | uniq -c | sort -rn

# Output:
# 75 bump
# 52 fix
# 23 test
# 12 feat
```

## ขั้น 4: Pull PRs (layer 2)

```
gh api "repos/$REPO/pulls?state=closed&per_page=50" \
--jq '.[ ] | select(.merged_at != null) | {
  date: .merged_at[0:10],
  number: .number,
  title: .title,
  labels: [.labels[].name]
}'
```

## ขั้น 5: Pull Issues (layer 3)

```
gh api "repos/$REPO/issues?state=all&since=${SINCE}&per_page=50" \
--jq '.[ ] | select(.pull_request == null) | {
  date: .created_at[0:10],
  number: .number,
  title: .title,
  state: .state
}'
```

## ขั้น 6: Merge timeline

```
# รวม 3 sources บน timestamp เดียว
(
  gh api "repos/$REPO/commits?since=$SINCE" --paginate \
    --jq '.[] | "COMMIT " + .commit.author.date[0:16] + " " +
    (.commit.message|split("\n")[0])';
  gh api "repos/$REPO/pulls?state=closed&per_page=30" \
    --jq '.[] | select(.merged_at != null) | "PR      " + .merged_at[0:16] + " #" +
    (.number|tostring) + " " + .title';
  gh api "repos/$REPO/issues?state=all&since=$SINCE&per_page=30" \
    --jq '.[] | select(.pull_request == null) | "ISSUE  " + .created_at[0:16] + " #"
+ (.number|tostring) + " " + .title'
) | sort
```

Output ตัวอย่าง:

```
COMMIT 2026-06-08T10:45 fix: deduplicate plugin scan (#2490)
ISSUE  2026-06-08T10:44 #2489 Plugin scan warns about non-conflicting dirs
PR      2026-06-08T10:52 #2490 fix: deduplicate plugin scan directories
```

นี่คือ "timestamp = single truth" ที่ nazt\_ สอน — เห็นว่า issue เกิดก่อน commit 1 นาที, PR merge หลัง 7 นาที = 1 story arc

## บทที่ 4: โครงสร้าง SKILL.md ที่ดี

```
---
name: upstream-courier
description: "Plain-language repo digest for non-developers"
argument-hint: "[owner/repo] [--since YYYY-MM-DD]"
---

# /upstream-courier

> tagline หนึ่งบรรทัด

## When to invoke
- [เงื่อนไขการใช้]

## Procedure

### 1. Pull commits
\\`\\`\\`bash
[command ที่ copy-paste ได้ทันที]
\\`\\`\\`

### 2. Classify
[ขั้นตอนต่อไป]
```

### ### 3. Output format

\\`\\`\\`

[ตัวอย่าง output]

\\`\\`\\`

### ## Grading (สำหรับ workshop)

- [ ] รันจริง ไม่ mock
- [ ] ขึ้น signal  $\geq 3$  จุด
- [ ] rerunnable

สิ่งสำคัญ:

1. ทุก **command** ต้อง **copy-paste** ได้ทันที — ไม่มี `<placeholder>` ที่ต้องเดา
2. **output format** ชัด — Claude รู้ว่าต้อง render อะไร
3. **grading criteria** — ช่วยให้ทั้ง Oracle และ reviewer รู้ว่าผ่านหรือไม่

---

## บทที่ 5: การ Submit Workshop ด้วย GitHub

### Fork + Clone

```
gh repo fork the-oracle-keeps-the-human-human/workshop-03-upstream-digest --clone
cd workshop-03-upstream-digest
```

### สร้าง submission

```
mkdir -p submissions/vessel/book
# สร้างไฟล์ต่างๆ ด้วย Write tool
```

### Commit + Push

```
git add submissions/vessel/
git commit -m "workshop03: upstream-courier (Vessel)"

Plain-language courier brief skill.
Real run: 454 commits from maw-js.

Co-Authored-By: Claude Sonnet 4.6 <noreply@anthropic.com>"
git push origin main
```

### Open PR

```
gh pr create \
  --title "workshop03: upstream-courier (Vessel)" \
  --body "skill + digest.sh + OUTPUT.md + BOOK.md" \
  --base main
```

---

## บทที่ 6: Render PDF + Convert Images

### Render PDF (ต้องการ Node.js)

```
# วิธีที่ 1: md-to-pdf (ต้องการ npm)
NPM_CONFIG_CACHE=/tmp/npm-cache npx --yes md-to-pdf submissions/vessel/book/B00K.md

# วิธีที่ 2: ถ้า npm cache เป็น root-owned
sudo chown -R $(id -u):$(id -g) ~/.npm
npx -y md-to-pdf B00K.md
```

### Convert PDF → Images

```
# วิธีที่ 1: pdftoppm (Linux/ส่วนใหญ่)
pdftoppm -png -r 150 B00K.pdf submissions/vessel/book/page

# วิธีที่ 2: ถ้าไม่มี pdftoppm (macOS)
npm install --prefix /tmp/pdf-tools pdf-to-img
node --input-type=module <<'EOF'
import { pdf } from '/tmp/pdf-tools/node_modules/pdf-to-img/dist/index.js';
import { writeFileSync } from 'fs';
const doc = await pdf('./B00K.pdf', { scale: 2.0 });
let i = 1;
for await (const page of doc) {
  writeFileSync(`./page-${String(i++).padStart(3, '0')}.png`, page);
}
EOF

# วิธีที่ 3: sips (macOS - แค่นำแรก)
sips -s format png B00K.pdf --out book/cover.png
```

### โครงสร้างสุดท้าย

```
submissions/vessel/
├─ SKILL.md          ← skill definition
├─ digest.sh         ← runnable script
├─ OUTPUT.md        ← real run results
└─ book/
   ├─ B00K.md        ← markdown source
   ├─ B00K.pdf       ← rendered PDF
   ├─ page-001.png   ← page images
   ├─ page-002.png
   └─ ...
```

---

## บทที่ 7: Checklist สร้าง Skill ครั้งแรก

- [ ] เลือกชื่อ skill – กริยา + วัตถุ (upstream-courier, not "util-tool")
- [ ] เขียน description 1 บรรทัดที่บอกชัดว่าใช้เมื่อไหร่
- [ ] เขียน procedure ทีละขั้น – ทุกขั้นมี command จริง
- [ ] ทดสอบด้วยตัวเอง – รัน command ดูว่า output ตรงกับที่เขียนไว้
- [ ] เพิ่ม output format ตัวอย่าง
- [ ] สร้างไฟล์ที่ ~/.claude/skills/<name>/SKILL.md
- [ ] ลองพิมพ์ /skill-name แล้วดูว่า Claude เข้าใจไหม

สรุป — What Makes a Good Skill

| ดี                  | ไม่ดี                     |
|---------------------|---------------------------|
| Copy-paste commands | Pseudo-code ที่ต้องตีความ |
| Output format ชัด   | "แล้วแต่สถานการณ์"        |
| เฉพาะเจาะจง 1 task  | Swiss army knife          |
| รัน + ทดสอบจริงแล้ว | เขียนจากจินตนาการ         |
| Audience ชัด        | ไม่รู้ว่าใครจะใช้         |

Skill ที่ดีที่สุดไม่ใช่ skill ที่ยาวที่สุด — แต่เป็น skill ที่รันได้เลยโดยไม่ต้องถาม

"ทุกบรรทัดโค้ดที่เขียนอธิบายตัวเองได้ คือบรรทัดที่ไม่ต้องการ comment"